


**Tujuan besar praktikum**
P2 menjawab "bagaimana parameter dipelajari". P3 menjawab "bagaimana kita memastikan hasil belajar model dapat dipercaya pada data baru". Praktikum ini membuktikan keduanya melalui hitungan manual, kode, tabel, dan grafik.

## Capaian Praktikum

1. Menjelaskan perbedaan population, sample, sampling variation, dan sampling bias melalui simulasi Python.
2. Membentuk train, validation, dan test serta menjelaskan mengapa ketiganya tidak boleh memiliki fungsi yang sama.
3. Menghubungkan konsep P2 (vector, weight, intercept, loss, objective, gradient descent) dengan training klasifikasi sederhana.
4. Membaca grafik training-vs-validation untuk membedakan improvement training dengan generalization.
5. Menafsirkan bias dan variance melalui repeated training pada sample berbeda.
6. Menjelaskan bagaimana regularization mengubah objective menjadi data loss + penalty.
7. Menggunakan cross-validation untuk mengurangi ketergantungan pada satu split.
8. Menjelaskan perubahan precision, recall, dan F1 ketika threshold berubah, lalu melakukan evaluasi final pada test.

## Peta Konsep P2 → P3

| Tahap | Pertanyaan utama | Konsep |
|---|---|---|
| P2 | Bagaimana model belajar dari data? | X/y → weight/intercept → prediction → loss → objective → gradient descent |
| P3 awal | Apakah data dan evaluasi kita valid? | sampling → split → generalization → bias/variance |
| P3 tengah | Bagaimana mengendalikan model? | regularization → cross-validation |
| P3 akhir | Bagaimana menilai keputusan model? | threshold → confusion matrix → precision/recall/F1 → test |

## 1. Sampling: "Sendok Sup" dalam Python

Kita membuat population sintetis 10.000 kondisi mesin. Karena ini simulasi, kita mengetahui population sehingga fenomena sampling dapat diamati secara terkendali. Pada kasus nyata, population biasanya tidak terlihat lengkap.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
N = 10_000
x1 = rng.uniform(0, 1, N)    # indikator temperatur
x2 = rng.uniform(0, 1, N)    # indikator getaran

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

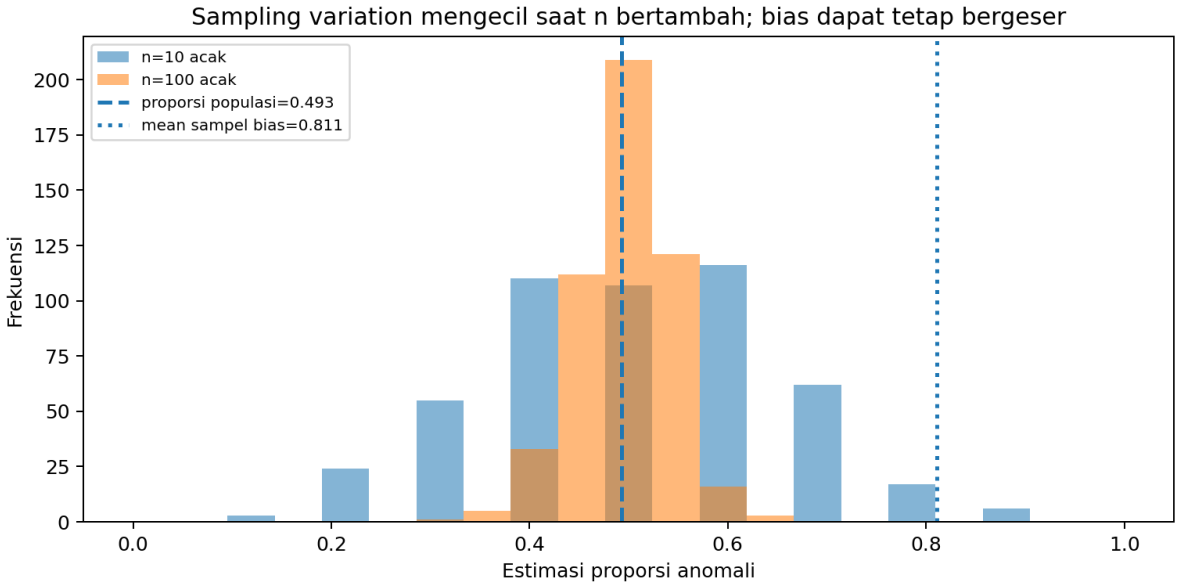
p_true = sigmoid(-4 + 5*x1 + 3*x2)
y_pop = rng.binomial(1, p_true)    # 0 normal, 1 anomali
print('Proporsi anomali population =', y_pop.mean())
```

**Hipotesis sebelum running**
Jika kita mengulang sampling acak, estimasi dari n=100 seharusnya lebih stabil daripada n=10. Tetapi sample besar tetap dapat bias jika mekanisme pemilihannya salah.

```
rng2 = np.random.default_rng(7)
prop10, prop100 = [], []
for _ in range(500):
    prop10.append(y_pop[rng2.choice(N, 10, replace=False)].mean())
    prop100.append(y_pop[rng2.choice(N, 100, replace=False)].mean())

biased_pool = np.where(x1 > 0.70)[0]
biased100 = []
for _ in range(200):
    biased100.append(y_pop[rng2.choice(biased_pool, 100, replace=False)].mean())

print('SD estimasi n=10  =', np.std(prop10))
print('SD estimasi n=100 =', np.std(prop100))
print('Mean sample bias   =', np.mean(biased100))
```




Gambar 1. Ukuran sample memperkecil sampling variation, tetapi mekanisme sampling yang bias tetap menggeser estimasi.

**Pertanyaan**
1) Bandingkan lebar distribusi n=10 dan n=100. 2) Mengapa garis mean sampel bias bergeser dari proporsi population? 3) Jelaskan mengapa “data banyak” tidak otomatis berarti “data representatif”.

## 2. Membentuk Train, Validation, dan Test

Ambil 240 observasi acak dari population sebagai dataset yang benar-benar “teramati”. Kita akan membaginya menjadi tiga peran. Test tidak boleh masuk ke training atau pemilihan model.

```
idx = rng.choice(N, size=240, replace=False)
X = np.column_stack([x1[idx], x2[idx]])
y = y_pop[idx].astype(float)

perm = rng.permutation(len(y))
X, y = X[perm], y[perm]

X_train, y_train = X[:144], y[:144]
X_val,   y_val   = X[144:192], y[144:192]
X_test,  y_test  = X[192:], y[192:]

print(X_train.shape, X_val.shape, X_test.shape)
print('proporsi kelas train/val/test =', y_train.mean(), y_val.mean(), y_test.mean())
```

| Subset | Dipakai untuk | Tidak boleh dipakai untuk |
|---|---|---|
| Training | menghitung gradient dan belajar w,b | laporan final generalization |
| Validation | memilih threshold / membantu pemilihan konfigurasi | mengubah w,b di loop training utama |

| | | |
|---|---|---|
| Test | evaluasi final setelah keputusan dikunci | memilih alpha, threshold, atau model |

**Pertanyaan**
Jika X_val dimasukkan ke gradient descent, apa fungsi validation yang hilang?

# 3. Training: Hubungkan P2 dengan Klasifikasi

## 3.1. Sigmoid

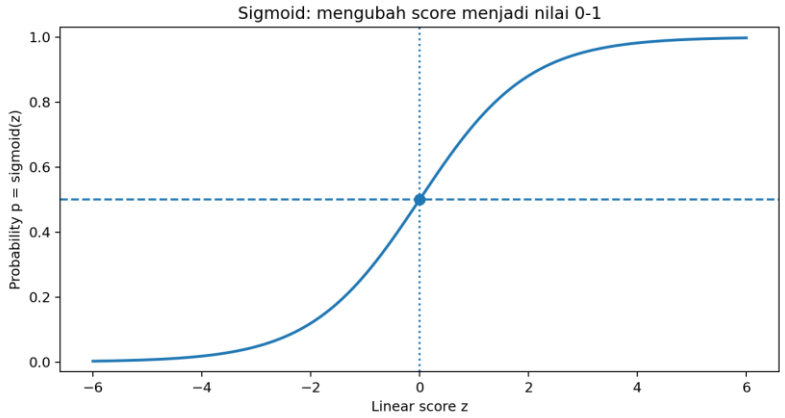
Pada P2, contoh utama menggunakan prediksi numerik/regresi. Model menghasilkan nilai langsung, misalnya energi 30,2 kWh. Untuk contoh klasifikasi biner, kita membutuhkan output yang dapat dibaca sebagai tingkat keyakinan kelas positif. Model linear terlebih dahulu menghasilkan score z, bukan probability.

$$z = b + w_1x_1 + w_2x_2$$

Score z dapat bernilai negatif maupun positif tanpa batas. Sigmoid mengubah score tersebut ke rentang 0 sampai 1:

$$p = \sigma(z) = 1 / (1 + e^{-z})$$

| z | sigmoid(z) | Intuisi |
|---|---|---|
| -4 | 0,018 | sangat condong ke kelas 0 |
| -1 | 0,269 | lebih condong ke kelas 0 |
| 0 | 0,500 | titik tengah |
| 1 | 0,731 | lebih condong ke kelas 1 |
| 4 | 0,982 | sangat condong ke kelas 1 |




Gambar 1. Sigmoid tidak memilih kelas; sigmoid hanya memetakan score z menjadi probability 0-1.

Kita menggunakan contoh deteksi kondisi mesin: kelas 0 = normal dan kelas 1 = anomali. Kita menggunakan model klasifikasi linear sederhana.

| Data | $x_1$: temperatur | $x_2$: getaran | Target y |
|---|---|---|---|
| A | 0,10 | 0,10 | 0 |
| B | 0,20 | 0,20 | 0 |
| C | 0,80 | 0,70 | 1 |
| D | 0,90 | 0,80 | 1 |

Feature sengaja sudah berada pada skala sekitar 0-1 agar hitungan manual mudah. Pada data nyata, scaling tetap harus mengikuti prinsip P2: parameter scaling dipelajari dari training data, bukan seluruh dataset.

X = [[0,1 0,1], [0,2 0,2], [0,8 0,7], [0,9 0,8]] ; y = [0, 0, 1, 1]

**Pengaitan dengan P2**
P2: z = b + x·w adalah dot product + intercept. Pada klasifikasi, z belum menjadi kelas. Sigmoid hanya memetakan score z menjadi probability. Gradient descent tetap memperbaiki weight dan intercept.

Parameter awal: w1=0, w2=0, b=0, learning rate η=1, alpha=0.1. Pada iterasi awal, z=0 untuk semua data sehingga sigmoid(0)=0.5.

| Data | y | z | p=sigmoid(z) | p−y |
|---|---|---|---|---|
| A | 0 | 0.0000 | 0.5000 | +0.5000 |
| B | 0 | 0.0000 | 0.5000 | +0.5000 |
| C | 1 | 0.0000 | 0.5000 | −0.5000 |
| D | 1 | 0.0000 | 0.5000 | −0.5000 |

Untuk target biner, kita memakai Binary Cross-Entropy (BCE). Konsepnya tetap sama dengan loss pada P2: memberi skor pada kesalahan. Fungsi loss berubah karena jenis task berubah dari prediksi numerik menjadi klasifikasi biner.

**Li = −[ yi ln(pi) + (1−yi) ln(1−pi) ]**

Pada p = 0,5, baik untuk y = 0 maupun y = 1, loss tiap data = 0,6931. Maka rata-rata data loss:

BCE = (0,6931 + 0,6931 + 0,6931 + 0,6931) / 4 = 0,6931

**Regularization Penalty = $(\alpha/2)(w_1^2 + w_2^2) = 0$**

**Objective J = BCE + Penalty** = 0,6931

Gradient weight terdiri dari dua bagian: sinyal dari data dan dorongan regularisasi.

**$\partial J/\partial w = (1/n)\, X^T(p-y) + \alpha w$**

**$\partial J/\partial b$ = mean(p-y)**

Untuk $w_1$:

[(0,5)(0,1) + (0,5)(0,2) + (-0,5)(0,8) + (-0,5)(0,9)] / 4 = -0,1750

Untuk $w_2$:

[(0,5)(0,1) + (0,5)(0,2) + (-0,5)(0,7) + (-0,5)(0,8)] / 4 = -0,1500

Karena weight masih nol, gradient regularisasi juga nol. Gradient intercept = 0 karena dua kelas seimbang pada titik awal.

$w_1$(new) = 0 - 1(-0,1750) = 0,1750

$w_2$(new) = 0 - 1(-0,1500) = 0,1500 ;

b(new) = 0

**Jembatan P2 → P3**
Di P2, contoh paling sederhana memakai Objective = Data Loss. P3 tidak mengganti konsep objective, tetapi memperluasnya: Objective = Data Loss + Regularization Penalty. Optimizer mengikuti objective total, bukan loss saja.

## Iterasi 1 - Gunakan bobot baru untuk memprediksi lagi

| Data | y | z | p | BCE per data |
|---|---|---|---|---|
| A | 0 | 0.0325 | 0.5081 | 0.7095 |
| B | 0 | 0.0650 | 0.5162 | 0.7262 |
| C | 1 | 0.2450 | 0.5609 | 0.5781 |
| D | 1 | 0.2775 | 0.5689 | 0.5640 |

$z_A$= 0 + (0,1 *0,175)+(0,1*0,150)

$p_A$= σ(z) = 1 / (1 + $e^{-0,0325}$)=0,5081

BCE = 0.6445

Penalty = $(0,1/2)(0,1750^2 + 0,1500^2)$ = 0.0027

Objective = 0.6445 + 0.0027 = 0.6471

Objective turun dari 0,6931 menjadi sekitar 0,6471. Ini adalah bukti bahwa satu update gradient descent bergerak ke arah yang lebih baik menurut objective yang kita definisikan.

| Komponen gradient | $w_1$ | $w_2$ |
|---|---|---|

| | | |
|---|---|---|
| Dari data | -0.1463 | -0.1245 |
| Dari regularisasi | 0.0175 | 0.0150 |
| Total gradient | -0.1288 | -0.1095 |

Update: w_new = w_old − η × gradient

$w_1$(new) = 0,1750 - (-0.1288) = 0.3038

$w_2$(new) = 0,1500 - (-0.1095) = 0.2595

b(new) = 0 - (0.0386) = -0.0386

### 3.3 Iterasi 2 dan 3 - Pola mulai terlihat

| Iterasi | $w_1$ | $w_2$ | b | BCE | Penalty | Objective |
|---|---|---|---|---|---|---|
| 0 | 0.0000 | 0.0000 | 0.0000 | 0.6931 | 0.0000 | 0.6931 |
| 1 | 0.1750 | 0.1500 | 0.0000 | 0.6445 | 0.0027 | 0.6471 |
| 2 | 0.3038 | 0.2595 | -0.0386 | 0.6119 | 0.0080 | 0.6199 |
| 3 | 0.4038 | 0.3441 | -0.0954 | 0.5869 | 0.0141 | 0.6010 |

Perhatikan dua hal. Pertama, data loss terus turun. Kedua, penalty justru naik karena weight membesar. Objective adalah kompromi keduanya. Ini adalah penghubung langsung antara objective P2 dan regularisasi P3.

**Kesimpulan yang benar**
"Objective turun, sehingga update parameter bergerak ke arah yang lebih baik menurut tujuan training. Penalty naik karena weight membesar, tetapi penurunan data loss masih lebih besar daripada kenaikan penalty."

**Kesimpulan yang kurang tepat**
"Karena objective turun, model pasti sudah bagus pada data baru." Training objective hanya menjelaskan kecocokan terhadap training objective. Generalisasi tetap harus dinilai dengan validation/test sesuai P3.

```
X_small = np.array([[0.10,0.10],[0.20,0.20],[0.80,0.70],[0.90,0.80]], float)
y_small = np.array([0,0,1,1], float)
eta, alpha = 1.0, 0.1
w = np.array([0.0, 0.0]); b = 0.0

def bce(y, p):
    p = np.clip(p, 1e-12, 1-1e-12)
    return -np.mean(y*np.log(p) + (1-y)*np.log(1-p))

def one_step(X, y, w, b, eta=1.0, alpha=0.1):
    z = b + X @ w
    p = sigmoid(z)
    data_loss = bce(y, p)
    reg = (alpha/2) * np.sum(w**2)
    objective = data_loss + reg
    diff = p - y
    grad_w = (X.T @ diff)/len(y) + alpha*w
    grad_b = np.mean(diff)
    return data_loss, reg, objective, grad_w, grad_b, w-eta*grad_w, b-eta*grad_b

for epoch in range(4):
    loss, reg, obj, gw, gb, w_new, b_new = one_step(X_small,y_small,w,b,eta,alpha)
    print(epoch,
'w=',w,'b=',round(b,4),'loss=',round(loss,4),'reg=',round(reg,4),'J=',round(obj,4))
    w, b = w_new, b_new
```

**Checkpoint wajib**
Python harus menghasilkan angka yang sejalan dengan hitungan manual. Jika berbeda, jangan lanjut sebelum memeriksa tanda p−y, learning rate, alpha, atau formula regularisasi.

**Analisis minimum**
Jangan menulis hanya "objective turun". Tulis: komponen apa yang turun, komponen apa yang naik, mengapa update weight menyebabkan perubahan probability, dan mengapa hal ini belum membuktikan generalization.

## 4. Generalization: Training Loss Turun Belum Cukup

Gunakan model mini yang sama, tetapi sekarang bandingkan training BCE dengan validation BCE. Validation sengaja memuat satu kasus noisy/kontradiktif agar efek “semakin yakin pada training” dapat terlihat.

```
X_val_small = np.array([
 [0.15,0.10],[0.25,0.30],[0.35,0.40],[0.75,0.60],[0.85,0.90],
 [0.45,0.30],[0.55,0.55],[0.65,0.45],[0.80,0.80]
], float)
y_val_small = np.array([0,0,1,1,1,0,1,1,0], float)

w = np.zeros(2); b = 0.0
train_hist, val_hist = [], []
for epoch in range(220):
    p_train = sigmoid(b + X_small @ w)
    p_val_s = sigmoid(b + X_val_small @ w)
    train_hist.append(bce(y_small, p_train))
    val_hist.append(bce(y_val_small, p_val_s))

    diff = p_train - y_small
    grad_w = (X_small.T @ diff)/len(y_small)  # alpha=0 untuk demonstrasi overfit
    grad_b = np.mean(diff)
    w -= 1.0*grad_w
    b -= 1.0*grad_b

best_epoch = int(np.argmin(val_hist))
print('validation minimum pada epoch', best_epoch)
```

```
# ==========================================
# GRAFIK Training vs validation loss
# ==========================================

epochs = np.arange(len(train_hist))

plt.figure(figsize=(10, 5))

plt.plot(
    epochs,
    train_hist,
    label='Training BCE'
)

plt.plot(
    epochs,
    val_hist,
    label='Validation BCE'
)

plt.axvline(
    best_epoch,
    linestyle='--',
    linewidth=2,
    label=f'validation minimum epoch {best_epoch}'
)

plt.xlabel('Epoch')
plt.ylabel('BCE')

plt.title(
    'Training loss dapat terus turun ketika '
    'validation loss mulai naik'
)

plt.legend()
plt.grid(alpha=0.2)

plt.show()
```

**Interpretasi yang diharapkan**
Sebelum titik minimum, perubahan parameter memperbaiki training dan validation. Setelah titik minimum, training masih membaik tetapi validation memburuk. Ini menunjukkan bahwa "lebih bagus pada training" tidak identik dengan "lebih baik pada data baru".

**Pertanyaan 1**
1) Pada epoch berapa validation BCE minimum?
2) Apa indikasi overfitting setelah titik tersebut?
3) Mengapa training objective saja tidak cukup untuk klaim generalization?

## 5. Bias–Variance melalui Repeated Training

Bias/variance tidak dapat disimpulkan dari satu run. Kita perlu mengulang training pada sample berbeda. Agar pola sangat terlihat, gunakan contoh regresi polynomial sederhana; fokus bagian ini adalah perilaku model, bukan mempelajari algoritma baru.

```
def f_true(x):
    return 4 + 1.2*x**2

rng_bv = np.random.default_rng(42)

def repeated_prediction(degree, repeats=100, n=20, x_star=2.0):
    preds = []
    for _ in range(repeats):
        x = rng_bv.uniform(-3, 3, n)
        y = f_true(x) + rng_bv.normal(0, 2.0, n)
        coef = np.polyfit(x, y, degree)
        preds.append(np.polyval(coef, x_star))
    return np.array(preds)

pred_d1 = repeated_prediction(1)
pred_d3 = repeated_prediction(3)
pred_d7 = repeated_prediction(7)
print('nilai acuan f(2) =', f_true(2.0))
print('SD degree 1/3/7 =', np.std(pred_d1), np.std(pred_d3), np.std(pred_d7))
# ==========================================
# GRAFIK Bias-Variance
# ==========================================

true_value = f_true(2.0)

plt.figure(figsize=(9, 5))

plt.boxplot(
    [pred_d1, pred_d3, pred_d7],
    tick_labels=['degree 1', 'degree 3', 'degree 7']
)

plt.axhline(
    true_value,
    linestyle='--',
    linewidth=2,
    label=f'nilai acuan f(2)={true_value:.1f}'
)

plt.ylabel('Prediksi pada x=2')

plt.title(
    'Bias-variance terlihat saat training '
    'diulang pada sampel berbeda'
)

plt.legend()
plt.grid(axis='y', alpha=0.2)

plt.show()
```

**Cara membaca grafik**
Posisi pusat box terhadap garis acuan memberi indikasi kesalahan sistematis (bias). Lebar/sebaran box menunjukkan sensitivitas terhadap sample training (variance). Jangan menyimpulkan variance dari satu run.

**Pertanyaan 2**
1.Bandingkan degree 1, 3, dan 7: mana yang paling stabil?

2.mana yang paling dekat dengan nilai acuan?

3.mana yang paling sensitif terhadap perubahan sample?

## 6. Regularization: Mengubah Objective, Bukan Mengganti Optimizer

Sekarang kembali ke model klasifikasi mini. Ubah alpha, tetapi pertahankan algoritma gradient descent. Dengan begitu mahasiswa dapat melihat bahwa regularization bekerja melalui objective dan gradient.

```
def train_simple(X, y, alpha, eta=0.1, epochs=500):
    w = np.zeros(X.shape[1]); b = 0.0
    for _ in range(epochs):
        p = sigmoid(b + X @ w)
        diff = p - y
        grad_w = (X.T @ diff)/len(y) + alpha*w
        grad_b = np.mean(diff)
        w -= eta*grad_w
        b -= eta*grad_b
    return w, b

alpha_values = [0, 0.01,0.05,0.10, 0.50, 1.00]

train_losses = []
val_losses = []
weight_norms = []

for alpha in alpha_values:

    w_a, b_a = train_simple( X_small, y_small, alpha    )

    # probability training
    p_train = sigmoid( b_a + X_small @ w_a )

    # probability validation
    p_val_a = sigmoid(b_a + X_val_small @ w_a)

    # loss
    train_loss = bce(y_small,p_train)

    val_loss = bce( y_val_small, p_val_a )

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    # ukuran besar seluruh weight
    weight_norms.append( np.linalg.norm(w_a) )

print('alpha          =', alpha_values)
print('training BCE   =', np.round(train_losses, 4))
print('validation BCE =', np.round(val_losses, 4))
print('||w||          =', np.round(weight_norms, 4))
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(
    alpha_values,
    train_losses,
    marker='o',
    label='Training BCE'
)

plt.plot(
    alpha_values,
    val_losses,
    marker='o',
    label='Validation BCE'
)

# symlog memungkinkan alpha=0 tetap ditampilkan,
# tetapi nilai lain tetap terlihat seperti skala log
plt.xscale(
    'symlog',
    linthresh=0.005
)

plt.xlabel('alpha regularisasi')
plt.ylabel('BCE')

plt.title(
    'Regularisasi mengubah trade-off '
    'training fit dan validation performance'
)

plt.legend()
plt.grid(alpha=0.2)

plt.show()
```

```
plt.figure(figsize=(9, 4))

plt.plot(
    alpha_values,
    weight_norms,
    marker='o'
)

plt.xscale(
    'symlog',
    linthresh=0.005
)

plt.xlabel('alpha regularisasi')
plt.ylabel('||w||')

plt.title(
    'Semakin kuat regularisasi, '
    'weight cenderung semakin kecil'
)

plt.grid(alpha=0.2)

plt.show()
```

**Hubungan langsung dengan teori**
P2: Objective = Data Loss. P3: Objective = Data Loss + α × Penalty. Gradient descent tetap sama prinsipnya, tetapi gradien sekarang memiliki tambahan αw yang mendorong weight lebih terkendali.

**Pertanyaan 3**
1) Apa yang terjadi pada ||w|| saat alpha naik?
2) Mengapa training BCE dapat naik?
3) Apakah validation BCE selalu membaik ketika alpha diperbesar?
4) Hubungkan alpha terlalu besar dengan underfitting.

## 7. Cross-Validation: Jangan Percaya Satu Split

Gunakan hanya training set 60% untuk memilih alpha dengan 5-fold cross-validation. Validation 20% tetap disimpan untuk pemilihan threshold, dan test tetap tertutup.

```
# X_train, y_train berasal dari Bagian 2
folds = np.array_split(np.arange(len(y_train)), 5)
alpha_candidates = [0, 0.01, 0.05, 0.10]
cv_result = []

for alpha in alpha_candidates:
    f1_fold = []
    for k in range(5):
        idx_val_fold = folds[k]
        idx_train_fold = np.concatenate([folds[j] for j in range(5) if j != k])

        w_cv, b_cv = train_simple(X_train[idx_train_fold], y_train[idx_train_fold],
alpha)
        p_cv = sigmoid(b_cv + X_train[idx_val_fold] @ w_cv)
        pred = (p_cv >= 0.5).astype(int)

        TP = np.sum((pred==1)&(y_train[idx_val_fold]==1))
        FP = np.sum((pred==1)&(y_train[idx_val_fold]==0))
        FN = np.sum((pred==0)&(y_train[idx_val_fold]==1))
        precision = TP/(TP+FP) if (TP+FP) else 0
        recall = TP/(TP+FN) if (TP+FN) else 0
        f1 = 2*precision*recall/(precision+recall) if precision+recall else 0
        f1_fold.append(f1)

    cv_result.append([alpha, np.mean(f1_fold), np.std(f1_fold)])
    print(alpha, np.round(f1_fold,3), 'mean=', round(np.mean(f1_fold),3), 'sd=',
round(np.std(f1_fold),3))
```

```
# ==========================================
# GRAFIK  Cross-validation
# ==========================================

cv_result_array = np.array(
    cv_result,
    dtype=float
)

alpha_cv = cv_result_array[:, 0]
mean_f1 = cv_result_array[:, 1]
std_f1 = cv_result_array[:, 2]

plt.figure(figsize=(10, 5))

plt.errorbar(
    alpha_cv,
    mean_f1,
    yerr=std_f1,
    marker='o',
    capsize=5
)

plt.xlabel('alpha')
plt.ylabel('Mean F1 ± SD')

plt.ylim(0, 1)
```

```
plt.title(
    'Cross-validation: baca mean dan '
    'variasi antar-fold bersama-sama'
)

plt.grid(alpha=0.2)

plt.show()
```

**Interpretasi**
Satu split dapat kebetulan menguntungkan atau merugikan suatu konfigurasi. CV mengurangi ketergantungan pada satu split, tetapi tidak memperbaiki sampling bias pada dataset awal.

**Pertanyaan 4**
Pilih alpha dari mean F1 cross-validation tertinggi. Jika dua alpha hampir sama, gunakan SD antar-fold sebagai informasi tambahan dan jelaskan keputusan.

## 8. Threshold, Precision, Recall, F1, dan Final Test

Setelah alpha dipilih dari training/CV, train satu model pada seluruh training set. Validation dipakai untuk memilih threshold. Threshold tidak mengubah bobot; threshold hanya mengubah probability menjadi kelas 0/1.

```
thresholds = np.arange(0.30, 0.71, 0.05)

precision_list = []
recall_list = []
f1_list = []

for t in thresholds:

    pred = (p_val >= t).astype(int)

    TP = np.sum((pred == 1) & (y_val == 1) )

    FP = np.sum( (pred == 1) & (y_val == 0) )

    FN = np.sum( (pred == 0) & (y_val == 1) )

    TN = np.sum( (pred == 0) & (y_val == 0) )

    precision = (TP / (TP + FP) if (TP + FP) > 0  else 0 )

    recall = (  TP / (TP + FN) if (TP + FN) > 0  else 0 )

    f1 = (2 * precision * recall / (precision + recall)
          if (precision + recall) > 0 else 0 )

    precision_list.append( precision )

    recall_list.append( recall )

    f1_list.append( f1 )

    print(
        f't={t:.2f}',
        'TP=', TP,
        'FP=', FP,
        'FN=', FN,
        'TN=', TN,
        'P=', round(precision, 3),
        'R=', round(recall, 3),
        'F1=', round(f1, 3)    )
best_index = np.argmax(f1_list)

best_threshold = thresholds[best_index]

print(
```

```
    'Best validation F1 threshold =',
    round(best_threshold, 2)
)
```

```
#grafik probability vs threshold
# Urutkan data validation agar grafik mudah dibaca
order = np.argsort(p_val)

p_sorted = p_val[order]
y_sorted = y_val[order]

x_pos = np.arange(len(p_sorted))

plt.figure(figsize=(10, 5))

plt.scatter(    x_pos,    p_sorted,    s=60)

plt.axhline(    0.50,    linestyle='--',    label='threshold = 0.50')

plt.xlabel('Observasi validation')
plt.ylabel('Probability kelas 1')

plt.title('Probability model dan garis threshold')

plt.ylim(0, 1)

plt.legend()
plt.grid(alpha=0.2)

plt.show()
```

```
# ============================================
# GRAFIK  Threshold vs Precision, Recall, F1
# ============================================
plt.figure(figsize=(10, 5))

plt.plot(
    thresholds,
    precision_list,
    marker='o',
    label='Precision'
)

plt.plot(
    thresholds,
    recall_list,
    marker='o',
    label='Recall'
)

plt.plot(
    thresholds,
    f1_list,
    marker='o',
    label='F1'
)

plt.axvline(
    best_threshold,
    linestyle='--',
    linewidth=2,
    label=(
        f'best F1 threshold='
        f'{best_threshold:.2f}'
    )
)

plt.xlabel('Threshold')
plt.ylabel('Metric')
```

```
plt.ylim(0, 1.05)

plt.title(    'Threshold mengubah precision, recall, '
    'dan F1 tanpa mengubah bobot model'
)

plt.legend()
plt.grid(alpha=0.2)

plt.show()
```

| Metrik | Pertanyaan yang dijawab |
|---|---|
| Precision | Dari semua yang diprediksi positif, berapa yang benar-benar positif? |
| Recall | Dari semua positive aktual, berapa yang berhasil ditemukan? |
| F1 | Bagaimana menyeimbangkan precision dan recall dalam satu angka? |
| Cost FP+10×FN | Bagaimana jika false negative jauh lebih mahal daripada false positive? |

**Pilih threshold menggunakan validation**
Pilih threshold sesuai protokol. Jangan gunakan test untuk mencari threshold yang membuat angka terlihat lebih baik.

Setelah threshold dikunci, buka test sekali:

```
threshold_final = 0.50  # GANTI dengan threshold pilihan validation
p_test = sigmoid(b_final + X_test @ w_final)
pred_test = (p_test >= threshold_final).astype(int)

TP = np.sum((pred_test==1)&(y_test==1))
FP = np.sum((pred_test==1)&(y_test==0))
FN = np.sum((pred_test==0)&(y_test==1))
TN = np.sum((pred_test==0)&(y_test==0))
precision = TP/(TP+FP) if (TP+FP) else 0
recall = TP/(TP+FN) if (TP+FN) else 0
f1 = 2*precision*recall/(precision+recall) if precision+recall else 0
print('TEST:', TP,FP,FN,TN, precision, recall, f1)
```

**Pertanyaan 5**
Jika test F1 lebih buruk daripada validation F1, bolehkah kita mengganti alpha atau threshold menggunakan test? Jelaskan dengan konsep fungsi epistemik test.

## 9. Sintesis dan Laporan Analisis

**1.** Notebook yang dapat dijalankan dari atas ke bawah tanpa error.

**2.** Semua grafik: sampling, training-vs-validation, bias-variance, regularization, cross-validation, threshold metrics.

**3.** Tabel keputusan alpha dan threshold beserta alasan.

**4.** Confusion matrix dan precision/recall/F1 pada validation dan final test.

**5.** Analisis 350–500 kata memakai pola Fakta → Mekanisme → Makna ML → Batas Kesimpulan.

**Kalimat kunci Pertemuan 3**
Model yang dapat dipercaya bukan model yang sekadar mempunyai training score bagus, tetapi model yang dievaluasi dengan sample, split, validasi, metric, dan protokol eksperimen yang sesuai sehingga klaim generalization memiliki bukti.